

Assignment :- 6
Parallel and Distributed Computing
Submitted by
Aunish Kumar Yadav (102303306)

Q1: Write the device query code, compile and run it on your system. Query enough information to know all the details of your device. Example queries: GPU card's name, GPU computation capabilities, Maximum number of block dimensions, Maximum number of grid dimensions, Maximum size of GPU memory, Amount of constant and share memory, Warp size, etc. Answer the following questions in your report.

1. What is the architecture and compute capability of your GPU?
2. What are the maximum block dimensions for your GPU?
3. Suppose you are launching a one dimensional grid and block. If the hardware's maximum grid dimension is 65535 and the maximum block dimension is 512, what is the maximum number threads can be launched on the GPU?
4. Under what conditions might a programmer choose not want to launch the maximum number of threads?
5. What can limit a program from launching the maximum number of threads on a GPU?
6. What is shared memory? How much shared memory is on your GPU?
7. What is global memory? How much global memory is on your GPU?
8. What is constant memory? How much constant memory is on your GPU?
9. What does warp size signify on a GPU? What is your GPU's warp size?
10. Is double precision supported on your GPU?

Code:

```
LAB6 > g++ dq.cu > main()
1  #include <stdio.h>
2  #include <cuda_runtime.h>
3
4  int main() {
5      int deviceCount;
6      cudaGetDeviceCount(&deviceCount);
7
8      printf("Number of CUDA devices: %d\n\n", deviceCount);
9
10     for (int i = 0; i < deviceCount; i++) {
11         cudaDeviceProp prop;
12         cudaGetDeviceProperties(&prop, i);
13
14         printf("Device %d: %s\n", i, prop.name);
15         printf("Compute Capability: %d.%d\n", prop.major, prop.minor);
16         printf("Total Global Memory: %lu bytes (%.2f GB)\n",
17             prop.totalGlobalMem,
18             prop.totalGlobalMem / (1024.0 * 1024 * 1024));
19
20         printf("Shared Memory per Block: %lu bytes\n", prop.sharedMemPerBlock);
21         printf("Constant Memory: %lu bytes\n", prop.totalConstMem);
22         printf("Warp Size: %d\n", prop.warpSize);
23
24         printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);
25         printf("Max Block Dimensions: (%d, %d, %d)\n",
26             prop.maxThreadsDim[0],
27             prop.maxThreadsDim[1],
28             prop.maxThreadsDim[2]);
29
30         printf("Max Grid Dimensions: (%d, %d, %d)\n",
31             prop.maxGridSize[0],
32             prop.maxGridSize[1],
33             prop.maxGridSize[2]);
34
35         printf("Registers per Block: %d\n", prop.regsPerBlock);
36
37         printf("Multiprocessor Count: %d\n", prop.multiProcessorCount);
38
39         printf("Double Precision Support: %s\n",
40             (prop.major >= 1 && prop.minor >= 3) ? "Yes" : "No");
41
42         printf("-----\n");
43     }
44
45     return 0;
46 }
```

Output:

```
aunish@LAPTOP-M6PM0AAP:~/UCS645/LAB6$ nvcc dq.cu -o device_query
./device_query
Number of CUDA devices: 1

Device 0: NVIDIA GeForce RTX 3050 Laptop GPU
Compute Capability: 8.6
Total Global Memory: 4294443008 bytes (4.00 GB)
Shared Memory per Block: 49152 bytes
Constant Memory: 65536 bytes
Warp Size: 32
Max Threads per Block: 1024
Max Block Dimensions: (1024, 1024, 64)
Max Grid Dimensions: (2147483647, 65535, 65535)
Registers per Block: 65536
Multiprocessor Count: 16
Double Precision Support: Yes
```

Answers to Asked Question :

Answer 1.

Architecture:

Ampere

Compute Capability: 8.6

Answer 2.

Maximum block dimensions:
(1024, 1024, 64)

Answer 3.

Maximum number of threads =
 $65535 \times 512 = 33,553,920$ threads

Answer 4.

A programmer may avoid launching the maximum number of threads when:

- The dataset is smaller than the maximum thread capacity
 - Each thread uses high registers or shared memory
 - To prevent resource contention and performance degradation
 - To improve memory access efficiency and cache usage
-

Answer 5.

Factors that limit maximum threads:

- Register usage per thread
 - Shared memory per block
 - Hardware occupancy limits
 - Kernel complexity and resource demands
-

Answer 6.

Shared Memory:

- Fast on-chip memory shared within threads of the same block
- Used for efficient communication and data reuse

Size: 49,152 bytes (48 KB)

Answer 7.

Global Memory:

- Large memory accessible by all threads but slower than shared memory

Size: 4.00 GB

Answer 8.

Constant Memory:

- Small read-only cached memory optimized for uniform access

Size: 65,536 bytes (64 KB)

Answer 9.

Warp Size:

- A group of threads executed together in SIMT fashion

Warp Size: 32 threads

Answer 10.

Yes, double precision is supported on this GPU.

Q2: Write a CUDA program to calculate the sum of the elements in an array. The array contains single precision floating point numbers. Generate your input array. For this question, do the following.

1. Allocate device memory
2. Copy host memory to device
3. Initialize thread block and kernel grid dimensions
4. Invoke CUDA kernel
5. Copy results from device to host
6. Free device memory
7. Write the CUDA kernel that computes the sum.

Code:

```
86 > array_sum.cu > main()
1  #include <iostream>
2  #include <vector>
3  #include <cuda_runtime.h>
4
5
6  __global__ void sumKernel(float* d_array, float* d_sum, int N) {
7      int idx = blockIdx.x * blockDim.x + threadIdx.x;
8      if (idx < N) {
9          atomicAdd(d_sum, d_array[idx]);
10     }
11 }
12
13 int main() {
14     int N = 10000;
15     size_t size = N * sizeof(float);
16
17
18     std::vector<float> h_array(N, 1.5f);
19     float h_sum = 0.0f;
20
21     float *d_array, *d_sum;
22     cudaMalloc((void**)&d_array, size);
23     cudaMalloc((void**)&d_sum, sizeof(float));
24
25     cudaMemset(d_sum, 0, sizeof(float));
26
27     cudaMemcpy(d_array, h_array.data(), size, cudaMemcpyHostToDevice);
28
29     int threadsPerBlock = 256;
30     int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
31
32     sumKernel<<<blocksPerGrid, threadsPerBlock>>>>(d_array, d_sum, N);
33     cudaDeviceSynchronize();
34     cudaMemcpy(&h_sum, d_sum, sizeof(float), cudaMemcpyDeviceToHost);
35
36     std::cout << "Total Sum: " << h_sum << std::endl;
37     cudaFree(d_array);
38     cudaFree(d_sum);
39
40     return 0;
41 }
```

Output:

```
● aunish@LAPTOP-M6PM0AAP:~/UCS645/LAB6$ nvcc array_sum.cu -o sum
● aunish@LAPTOP-M6PM0AAP:~/UCS645/LAB6$ ./sum
Total Sum: 15000
```

Q3: Perform Matrix Addition of two large integer matrices in CUDA.

Answer the following questions.

1. How many floating operations are being performed in the matrix addition kernel?
2. How many global memory reads are being performed by your kernel?
3. How many global memory writes are being performed by your kernel?

Code:

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>

__global__ void matAddKernel(int* A, int* B, int* C, int width, int height) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (col < width && row < height) {
        int idx = row * width + col;
        C[idx] = A[idx] + B[idx];
    }
}

int main() {
    int width = 1024, height = 1024;
    size_t size = width * height * sizeof(int);

    std::vector<int> h_A(width * height, 1);
    std::vector<int> h_B(width * height, 2);
    std::vector<int> h_C(width * height, 0);

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A.data(), size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((width + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (height + threadsPerBlock.y - 1) / threadsPerBlock.y);

    matAddKernel<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, width, height);

    cudaMemcpy(h_C.data(), d_C, size, cudaMemcpyDeviceToHost);

    std::cout << "Top-left element of result matrix: " << h_C[0] << " (Expected 3)" << std::endl;

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

Output:

```
● aunish@LAPTOP-M6PM0AAP:~/UCS645/LAB6$ nvcc matrix_add.cu -o ma
● aunish@LAPTOP-M6PM0AAP:~/UCS645/LAB6$ ./ma
Top-left element of result matrix: 3 (Expected 3)
```

Answers to asked Questions:

1. **Floating operations:** Zero, Because the prompt specified "two large *integer* matrices," the arithmetic operation ($A[idx] + B[idx]$) uses integer addition, not floating-point operations (FLOP's).
2. **Global memory reads:** For an $N \times M$ matrix, every thread reads one element from matrix A and one element from matrix B.
Total reads = $2 \times N \times M$.
3. **Global memory writes:** Every thread writes one computed element to the result matrix C.
Total writes = $N \times M$.